TAREA M28 – CD-YAIR PALOMARES

- Extrae en línea la base de datos "Iris.csv" de la siguiente liga y agrégala a un DataFrame en Python.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

os.chdir("C:/Users/yair_/OneDrive/Desktop/Ciencia de Datos/Módulo 7 - DataFrames")
data = pd.read_csv("iris.csv")
data
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

```
#Preprocesado y modelado
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from numpy import linalg as LA
```

- Efectúa un análisis de componentes principales (PCA) para los datos de la base de Iris y comenta sobre cuál sería el grado de explicación de la varianza que tendría al utilizar los dos primeros componentes.

```
#Determinar el número de renglones de la base de datos
index = data.index
renglones = len(index)

#Estandarización de las variables
df2 = data.drop("variety", axis=1)
df2 = StandardScaler().fit_transform(df2)
df2
```

```
array([[-9.00681170e-01,  1.01900435e+00, -1.34022653e+00,
        -1.31544430e+00],
       [-1.14301691e+00, -1.31979479e-01, -1.34022653e+00,
        -1.31544430e+00],
       [-1.38535265e+00,  3.28414053e-01, -1.39706395e+00,
        -1.31544430e+00],
       [-1.50652052e+00,  9.82172869e-02, -1.28338910e+00,
        -1.31544430e+00],
       [-1.02184904e+00,  1.24920112e+00, -1.34022653e+00,
        -1.31544430e+00],
       [-5.37177559e-01,  1.93979142e+00, -1.16971425e+00,
        -1.05217993e+00],
       [-1.50652052e+00,  7.88807586e-01, -1.34022653e+00,
        -1.18381211e+00],
       [-1.02184904e+00,  7.88807586e-01, -1.28338910e+00,
        -1.31544430e+00],
       [-1.74885626e+00, -3.62176246e-01, -1.34022653e+00,
        -1.31544430e+00],
```

```
#Calcular la matriz de correlaciones para la matriz transformada
A = (1/renglones) * np.dot(df2.T, df2)
A
```

```
array([[ 1.        , -0.11756978,  0.87175378,  0.81794113],
       [-0.11756978,  1.        , -0.4284401 , -0.36612593],
       [ 0.87175378, -0.4284401 ,  1.        ,  0.96286543],
       [ 0.81794113, -0.36612593,  0.96286543,  1.        ]])
```

```python
#Entrenamiento del modelo PCA con escalado de los datos
pca_pipe = make_pipeline(PCA())
pca_pipe.fit(df2)

#Se extrae el modelo entrenado del pipeline
modelo_pca = pca_pipe.named_steps["pca"]

print("Eigenvalores:") #indicadores de explicación de varianza
results = LA.eigvals(A)
print(results)
```

```
Eigenvalores:
[2.91849782 0.91403047 0.14675688 0.02071484]
```

```python
#Porcentaje de la varianza explicada por cada nuevo componente
print("Porcentaje de varianza explicada por componente")
print(modelo_pca.explained_variance_ratio_)
```

```
Porcentaje de varianza explicada por componente
[0.72962445 0.22850762 0.03668922 0.00517871]
```

```python
#Grado de explicación de la varianza al utilizar los dos primeros componentes
print("Al utilizar los dos primeros componentes se puede explicar un ", (0.7296 + 0.2285)*100, "% de la varianza.")
```

```
Al utilizar los dos primeros componentes se puede explicar un  95.81 % de la varianza.
```

- Grafica tanto el mapa de observaciones como el de factores asociados a esta base de datos.
  - ¿Qué puede concluir al compararlos?
  - Corrobore sus conclusiones al revisar casos específicos en su Data Frame.

```python
# Cálculo de los eigenvectores
print("Eigenvectores (por renglón):")
pd.DataFrame(data = modelo_pca.components_,
             columns = ["sepal.length", "sepal.width", "petal.length", "petal.width"],
             index = ["PC1", "PC2", "PC3", "PC4"])
```

```
Eigenvectores (por renglón):
```

	sepal.length	sepal.width	petal.length	petal.width
PC1	0.521066	-0.269347	0.580413	0.564857
PC2	0.377418	0.923296	0.024492	0.066942
PC3	-0.719566	0.244382	0.142126	0.634273
PC4	-0.261286	0.123510	0.801449	-0.523597

```python
#Proyecciones de los componentes
proyecciones = np.dot(modelo_pca.components_, df2.T)
proyecciones = pd.DataFrame(proyecciones, index = ["PC1", "PC2", "PC3", "PC4"])
proyecciones = proyecciones.transpose().set_index(data.index)
proyecciones
```

	PC1	PC2	PC3	PC4
0	-2.264703	0.480027	-0.127706	-0.024168
1	-2.080961	-0.674134	-0.234609	-0.103007
2	-2.364229	-0.341908	0.044201	-0.028377
3	-2.299384	-0.597395	0.091290	0.065956
4	-2.389842	0.646835	0.015738	0.035923
...	...	...	...	...
145	1.870503	0.386966	0.256274	-0.389257
146	1.564580	-0.896687	-0.026371	-0.220192
147	1.521170	0.269069	0.180178	-0.119171
148	1.372788	1.011254	0.933395	-0.026129
149	0.960656	-0.024332	0.528249	0.163078

150 rows × 4 columns

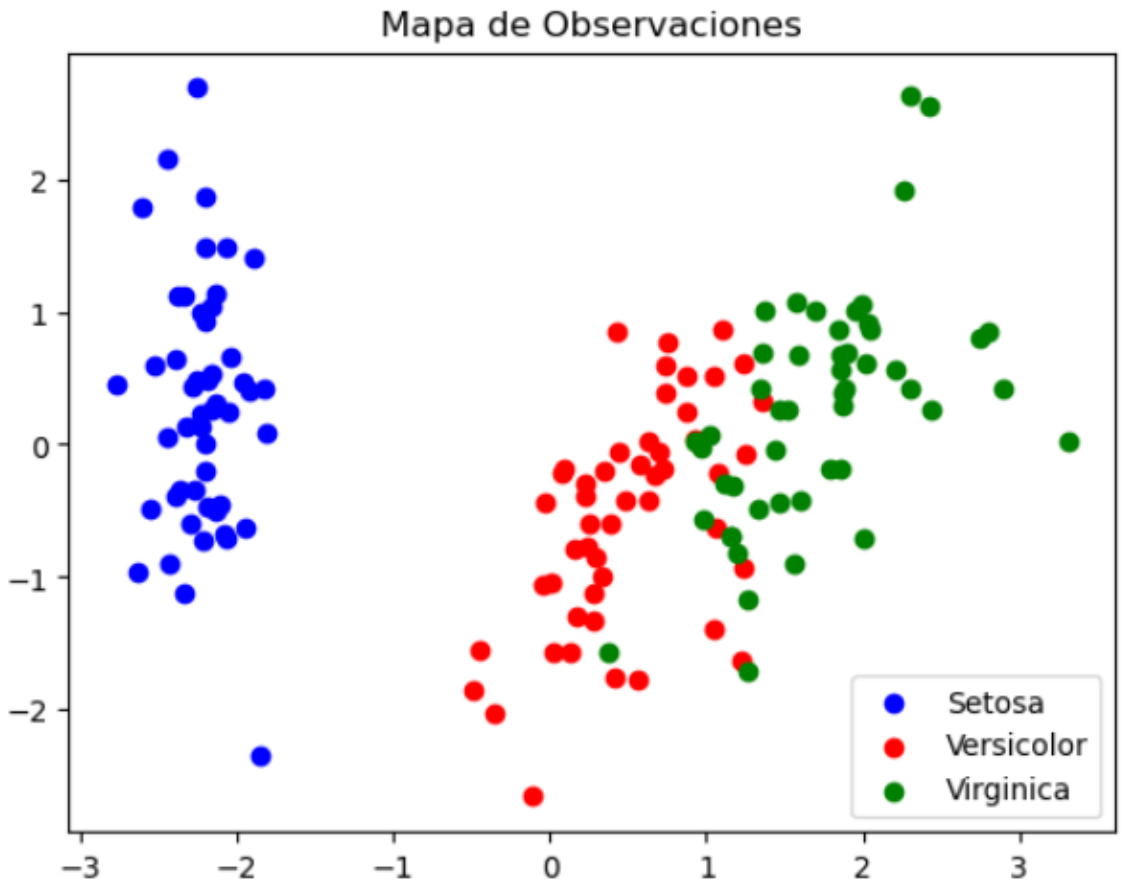
```python
# Mapa de observaciones
proyecciones = pd.concat([proyecciones, data["variety"]], axis=1)

x_setosa = proyecciones[proyecciones["variety"] == "Setosa"].iloc[:,0]
y_setosa = proyecciones[proyecciones["variety"] == "Setosa"].iloc[:,1]
x_setosa = x_setosa.to_numpy()
y_setosa = y_setosa.to_numpy()

x_versicolor = proyecciones[proyecciones["variety"] == "Versicolor"].iloc[:,0]
y_versicolor = proyecciones[proyecciones["variety"] == "Versicolor"].iloc[:,1]
x_versicolor = x_versicolor.to_numpy()
y_versicolor = y_versicolor.to_numpy()

x_virginica = proyecciones[proyecciones["variety"] == "Virginica"].iloc[:,0]
y_virginica = proyecciones[proyecciones["variety"] == "Virginica"].iloc[:,1]
x_virginica = x_virginica.to_numpy()
y_virginica = y_virginica.to_numpy()

fig, ax = plt.subplots()
ax.set_title("Mapa de Observaciones")
ax.scatter(x_setosa, y_setosa, color = "blue", label = "Setosa")
ax.scatter(x_versicolor, y_versicolor, color = "red", label = "Versicolor")
ax.scatter(x_virginica, y_virginica, color = "green", label = "Virginica")
plt.legend()
plt.show()
```



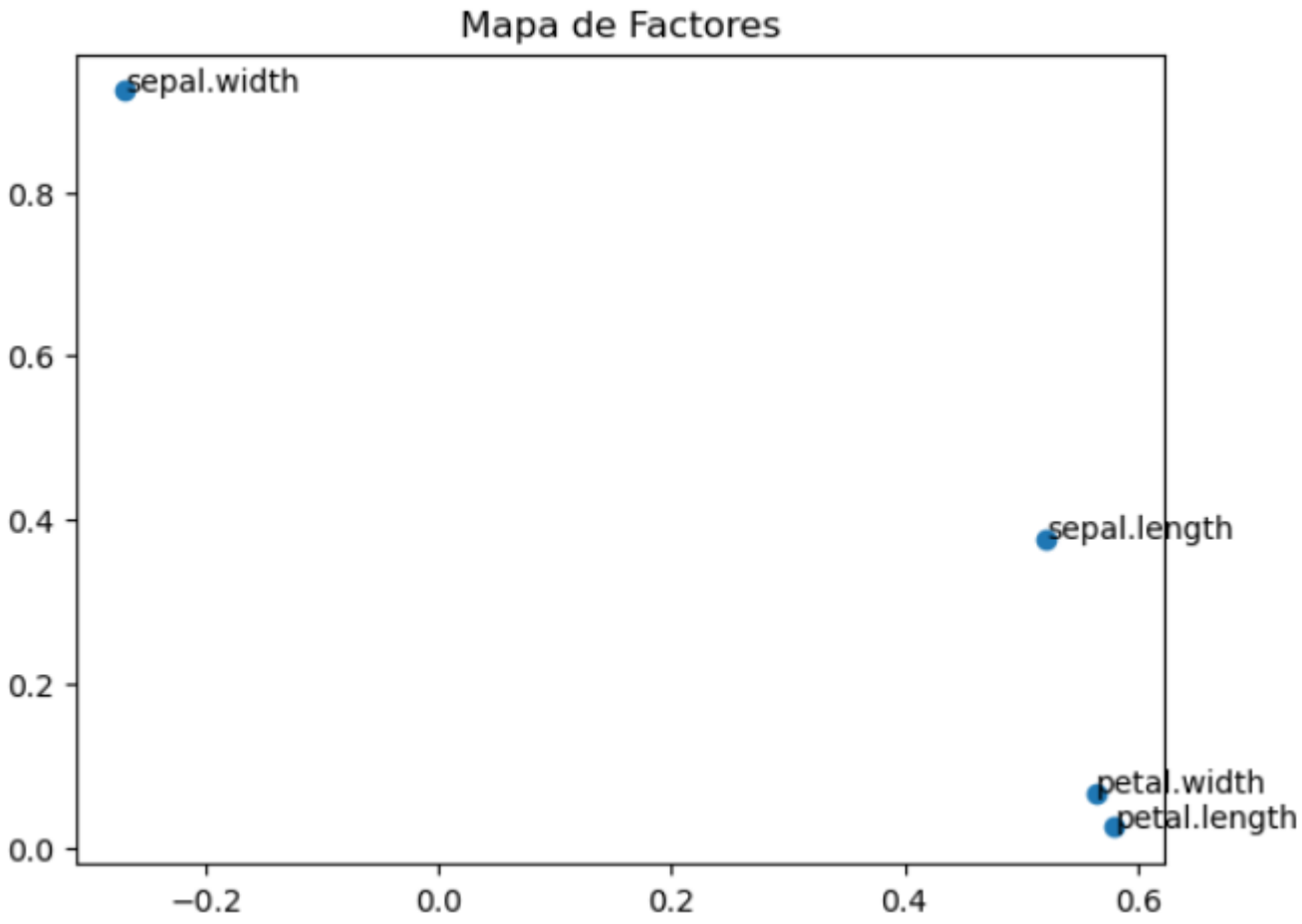

```
componentes_2 = pd.DataFrame(data = modelo_pca.components_,
                             columns = ["sepal.length", "sepal.width", "petal.length", "petal.width"],
                             index = ["PC1", "PC2", "PC3", "PC4"])
componentes_2 = componentes_2.iloc[0:2, :]
componentes_2 = componentes_2.T
componentes_2
```

	PC1	PC2
sepal.length	0.521066	0.377418
sepal.width	-0.269347	0.923296
petal.length	0.580413	0.024492
petal.width	0.564857	0.066942

```python
# Mapa de factores
x = componentes_2.iloc[:,0]
y = componentes_2.iloc[:,1]
z = componentes_2.index
x = x.to_numpy()
y = y.to_numpy()

fig, ax = plt.subplots()
ax.set_title("Mapa de Factores")
ax.scatter(x, y)

for i, txt in enumerate(z):
    ax.annotate(txt, (x[i], y[i]))
```



```
# ¿Qué puede concluir al compararlos?
print("Posterior al análisis PCA y la graficación de los mapas de observaciones y factores, se puede concluír que el factor característico de las setosas
print()
print("Por otro lado, también se puede concluir que la longitud del sépalo en las virginias es mayor a los otros pétalos estudiados.")
print()
print("Finalmente, otra de las deducciones que nos permite realizar el presente análisis de clasificación, es que los pétalos versicolores son los que pr
```

```
Posterior al análisis PCA y la graficación de los mapas de observaciones y factores, se puede concluír que el factor característico de las setosas es el
ancho del sépalo, ya que como se puede observar en los datos, este suele ser más grande que el resto de las especies de flores analizadas.

Por otro lado, también se puede concluir que la longitud del sépalo en las virginias es mayor a los otros pétalos estudiados.

Finalmente, otra de las deducciones que nos permite realizar el presente análisis de clasificación, es que los pétalos versicolores son los que presentan
mayor tamaño tanto en ancho como largo.
```